BLOCKCHAIN
CODE-ONLY EXAM PRACTICE

Likely coding tasks, complete answers, code walkthroughs, and how to run tests

Focus: Practical Python coding questions similar to the Blockchain Integrity Verification lab. No long theory-question section.

Inside: the full validator, smaller coding tasks, bug-fixing questions, nonce handling, tampering tests, and a step-by-step run guide.

Use this as a practice workbook: first try writing the code yourself, then compare your solution with the answer.

The code examples assume a dictionary-based block structure and a project with methods/constants such as read_from_disk(), hash256(), ZERO_HASH, and TARGET. Always use the exact names and hash serialization from the starter code in the exam.

1. The main lab question — complete code

Task: Implement validate_chain() to check the empty chain, genesis block, heights, previous hashes, recalculated hashes, and mining target.

```
def validate_chain(self):
    chain = self.read_from_disk()

    # An empty chain is invalid
    if not chain:
        return False

    # Validate the genesis block
    first_block = chain[0]
    first_header = first_block["BlockHeader"]

    if first_block["Height"] != 1:
        return False

    if first_header["prev_block_hash"] != ZERO_HASH:
        return False

    # Validate every block
    for index, block in enumerate(chain):
        header = block["BlockHeader"]

        # Height must match the block's one-based position
        if block["Height"] != index + 1:
            return False

        # Determine the expected previous-block hash
        if index == 0:
            expected_prev_hash = ZERO_HASH
        else:
            expected_prev_hash = (
                chain[index - 1]["BlockHeader"]["blockHash"]
            )

        if header["prev_block_hash"] != expected_prev_hash:
            return False

        # This version assumes mining increments nonce
        # immediately after hashing, including on success.
        nonce_used = header["nonce"] - 1

        hash_data = (
            str(header["version"])
            + header["prev_block_hash"]
            + header["merkle_root"]
            + str(header["timestamp"])
            + str(header["bits"])
            + str(nonce_used)
        ).encode("utf-8")

        calculated_hash = hash256(hash_data)

        if calculated_hash != header["blockHash"]:
            return False

        if not calculated_hash.startswith(TARGET):
            return False

    return True
```

Adapt before running: This is the dictionary-based version. If your starter code uses object attributes, replace expressions such as `block['Height']` with the matching attribute. The field order and hash function must exactly match the mining code.

What each code section does

Code pattern	Why it is there
if not chain	Rejects an empty chain before indexing it.
chain[0]	Gets the genesis block.
index + 1	Converts Python's zero-based index to a height starting at 1.
chain[index - 1]	Gets the preceding block for a non-genesis block.
nonce - 1	Recovers the nonce if mining increments it after calculating the successful hash.
hash256(hash_data)	Recalculates the hash from the header fields.
calculated_hash != stored hash	Detects a hash/data mismatch.
startswith(TARGET)	Checks the lab's prefix-based mining target.
return True after loop	Accepts only after all blocks pass.

2. Coding question — validate only block heights

Question: Return False if the chain is empty or if any block's height is not sequential.

```
def validate_heights(self):
    chain = self.read_from_disk()

    if not chain:
        return False

    for index, block in enumerate(chain):
        expected_height = index + 1

        if block["Height"] != expected_height:
            return False

    return True
```

Test mentally: heights [1, 2, 3] return True; heights [1, 2, 4] return False. The first mismatch ends the function immediately.

3. Coding question — validate previous-hash links

Question: The genesis block must point to ZERO_HASH. Each later block must point to the previous block's stored hash.

```
def validate_links(self):
    chain = self.read_from_disk()

    if not chain:
        return False

    for index, block in enumerate(chain):
        actual_prev_hash = block["BlockHeader"]["prev_block_hash"]

        if index == 0:
            expected_prev_hash = ZERO_HASH
        else:
            expected_prev_hash = (
                chain[index - 1]["BlockHeader"]["blockHash"]
            )

        if actual_prev_hash != expected_prev_hash:
            return False

    return True
```

This checks the links, not whether each block's stored hash is itself correct. That requires recalculating the hash separately.

4. Coding question — validate one block's hash

Question: Recalculate one block's hash and return whether it matches the stored hash.

```
def validate_block_hash(block):
    header = block["BlockHeader"]

    # Use -1 only if mining increments nonce after hashing
    nonce_used = header["nonce"] - 1

    hash_data = (
        str(header["version"])
        + header["prev_block_hash"]
        + header["merkle_root"]
        + str(header["timestamp"])
        + str(header["bits"])
        + str(nonce_used)
    ).encode("utf-8")

    calculated_hash = hash256(hash_data)

    return calculated_hash == header["blockHash"]
```

If mining increments the nonce only after a failed hash, use `header['nonce']` directly. The validator must match the miner's exact data serialization.

5. Coding question — validate the target

Question: Given a recalculated hash and a target prefix, return True if the hash starts with that prefix.

```
def validate_target(calculated_hash, target):
    return calculated_hash.startswith(target)
```

Target	Hash prefix	Result
0000	0000ab12...	True
0000	0001ab12...	False
abc	abcdef...	True
abc	0abcdef...	False

In the full validator, check the **recalculated** hash, not only the stored hash. This lab uses a string-prefix target; other systems may compare a numeric hash to a numeric target.

6. Coding question — modify a nonce and test tampering

Question: Test the original chain, modify one block's nonce, then test the chain again.

```
import copy

blockchain = Blockchain()

# Case 1: original chain
print("Case 1: Original blockchain")
print("Blockchain Valid:", blockchain.validate_chain())

# Make a backup and an independent copy
original_chain = blockchain.read_from_disk()
tampered_chain = copy.deepcopy(original_chain)

# Change the nonce in the copied chain
tampered_chain[0]["BlockHeader"]["nonce"] += 1

# Save the modified chain with the database's real write method.
# Replace save_chain with that method's actual name.
blockchain.save_chain(tampered_chain)

# Case 2: tampered chain
print("\nCase 2: After modifying a block's nonce")
print("Blockchain Valid:", blockchain.validate_chain())

# Restore the original chain after testing
blockchain.save_chain(original_chain)
```

Expected output:

```
Case 1: Original blockchain
Blockchain Valid: True

Case 2: After modifying a block's nonce
Blockchain Valid: False
```

Important: `save_chain()` is a placeholder because the database API differs by starter project. Find the actual method that writes the list of blocks, and use it in both save calls. Restore the original chain so the test does not leave your data altered.

7. Coding question — fix a bug in the loop

Question: Find why this code can accept an invalid chain.

```
def validate_heights(self):
    chain = self.read_from_disk()

    for index, block in enumerate(chain):
        if block["Height"] != index + 1:
            return False
        return True
```

Bug: `return True` is inside the loop. It can stop after checking the first block.

Correct answer:

```
def validate_heights(self):
    chain = self.read_from_disk()

    if not chain:
        return False

    for index, block in enumerate(chain):
        if block["Height"] != index + 1:
            return False

    return True
```

8. Coding question — combine the checks

Question: Write a validator that uses separate helper methods for height, links, hash integrity, and target.

```
def validate_chain(self):
    chain = self.read_from_disk()

    if not chain:
        return False

    if chain[0]["Height"] != 1:
        return False

    if chain[0]["BlockHeader"]["prev_block_hash"] != ZERO_HASH:
        return False

    for index, block in enumerate(chain):
        if block["Height"] != index + 1:
            return False

        if not self.validate_link_at(chain, index):
            return False

        calculated_hash = self.calculate_block_hash(block)

        if calculated_hash != block["BlockHeader"]["blockHash"]:
            return False

        if not calculated_hash.startswith(TARGET):
            return False

    return True
```

This version assumes you also implement `validate_link_at(chain, index)` and `calculate_block_hash(block)`. It is a modular design: each helper has one responsibility. Do not use this exact version if the exam asks for a single self-contained method.

9. How to put the code into the project

Step 1 — Open the project folder. Open the folder containing the blockchain package in VS Code or a terminal. Find `blockchain.py`, `blockheader.py`, and `database.py`.

Step 2 — Add the method. Put `validate_chain()` inside the `Blockchain` class at the same indentation level as its other methods. Do not put it inside another method.

Step 3 — Check imports. Ensure the file imports the hash function and constants from the project's actual modules. Names and import paths must match the starter code.

Step 4 — Add the demonstration. Put the two test cases in the project's existing entry-point file, or create a small test file that imports `Blockchain`. Do not accidentally mine extra blocks before testing.

Step 5 — Use the real database write method. The screenshot's task expects a modified nonce to be saved before validation reads the chain. Replace the example placeholder with the actual database API.

Typical terminal workflow

```
# 1. Open a terminal in the project root
cd path/to/project

# 2. If the project uses a virtual environment, activate it
# Linux/macOS:
source .venv/bin/activate

# Windows PowerShell:
.venv\Scripts\Activate.ps1

# 3. Run the existing entry point (if it is main.py)
python main.py
```

The exact command depends on the starter project's layout. Run the file that creates/loads the blockchain and calls the tests. If the package is designed to run as a module, use the project's documented `python -m package.module` command instead.

10. How to test outputs reliably

Test	Expected result	What it proves
Original untouched chain	True	Existing chain passes all checks.
Empty chain	False	Empty-chain guard works.
Genesis height changed	False	Genesis and height checks work.
Genesis previous hash changed	False	Genesis-link check works.
Later height changed	False	Sequential-height check works.
Later previous hash changed	False	Link check works.
Nonce changed but stored hash kept	False	Hash recalculation detects tampering.
Hash does not meet TARGET	False	Target check works.

For safe testing, work on a copy of the database or back up the original file first. Restore the original after tampering tests. Do not submit a screenshot of expected output typed manually; run the program and capture the real terminal output.

11. Common run errors and fixes

NameError: ZERO_HASH or TARGET is not defined

Import the constants from the configuration file supplied by the starter code. Do not guess the module path; inspect the existing imports.

KeyError: 'BlockHeader' or another field

The starter code may use different dictionary keys or class attributes. Print one block's structure or inspect the block class, then adjust the field access.

Calculated hash never matches

Check the exact field order, string conversions, encoding, hash algorithm, and nonce behavior. The validator must reproduce the mining input exactly.

ModuleNotFoundError

Run Python from the project root, activate the correct virtual environment, and use the import path intended by the starter code.

Original chain prints False

First verify that the database really contains a mined chain. Then check the genesis rules, height numbering, stored links, nonce convention, and target format.

Tampered chain still prints True

Ensure the modified chain was actually saved to the same database read by `read_from_disk()`, and ensure the validator recalculates the hash instead of trusting the stored value.

12. Last-minute code checklist

- Empty chain returns False.
- Genesis height is exactly 1.
- Genesis previous hash equals ZERO_HASH.
- Every height equals index + 1.
- Each later block points to the previous block's stored hash.
- Hash input matches the mining code exactly.
- Nonce handling matches the successful mining attempt.
- Calculated hash equals the stored hash.
- Calculated hash satisfies TARGET.
- return True is outside the loop.
- Both test cases are executed and real output is captured.
- Original database is restored after tampering tests.

Best exam strategy: Translate each requirement into one check. Implement the simplest correct version first, then test one failure case per check. The most common mistakes are incorrect indentation, using index instead of index + 1, checking the wrong previous block, and misreading the nonce increment.